

[toc]

库无关知识

Key

- [Python进阶——如何正确使用yield?](#)

生成器 `yield` 关键字介绍

- [彻底明白 Python partial\(\)](#)

`functools.partial` 偏函数类似于装饰器，可以扩展函数的功能。

“把一个函数的某些参数给固定住（也就是设置默认值），返回一个新的函数，调用这个新函数会更简单”，作用主要是减少调用func时要传递的参数个数。

- [IPython display How To Use "From IPython Import Display" In Your Projects](#)

IPython 中的模块的核心是用于显示对象的丰富表示形式。虽然 Python 的内置函数可以输出文本，但更进一步，渲染各种多媒体格式和复杂的视觉效果。

类

- [Python中的内部类 \(Inner Classes\)](#)

定义在某一个类的里面的类被称为内部类/嵌套类。目的是紧密把两个或者多个类打包在一起，可以用 `self` 关键词访问内部类，但不能在内部类访问外部类。

类修饰

- [@property](#)

[python @property的介绍与使用](#)

使用 `@property` 装饰器来创建只读属性，`@property` 装饰器会将方法转换为相同名称的只读属性，可以与所定义的属性配合使用，这样可以防止属性被修改。

```
class DataSet(object):
    def __init__(self):
        self._images = 1
        self._labels = 2 #定义属性的名称

    @property
    def images(self): #方法加入@property后，这个方法相当于一个属性，这个属性可以让用户进行使用，而且用户有没办法随意修改。
        return self._images

    @property
    def labels(self):
        return self._labels

l = DataSet()
#用户进行属性调用的时候，直接调用images即可，而不用知道属性名_images，因此用户无法更改属性，从而保护了类的属性。
print(l.images) # 加了@property后，可以用调用属性的形式来调用方法，后面不需要加()。
```

- **@store**

`brain-score` 项目自定义装饰器, [result_caching](#)

存储函数结果, 以便它们不会在具有相同参数的函数的重复调用上再次计算。结果可以存储在磁盘或内存中。

```
from result_caching import store
@store()
def f(a, b):
    return a * b
y = f(1, 2) # computed first time, stored on disk
y = f(1, 2) # not computed again, loaded from disk
y = f(1, 3) # computed again, different parameters
```

- **@classmethod**

[python @classmethod 的使用场合](#)

在重构类的时候不需要修改构造函数, 只要额外添加你要处理的函数, 然后使用[@classmethod](#)即可

格式

parquet: 一种专为大数据处理系统优化的列式存储文件格式, 由 Twitter 和 Cloudera 两个在大数据生态系统中具有影响力的公司 (曾经) 于 2013 年共同创建, 目标是开发一种高效, 高性能的列式存储格式, 并且能够与各种数据处理系统兼容。

代码库

NumPy

Pandas

先把pandas的[官方文档](#)放这儿

- **.loc .iloc**

pandas的loc为Selection by Label函数, 即为按标签取数据。第一个参数选择index, 第二个参数选择column。

iloc函数为Selection by Position, 即按位置选择数据, 即第n行, 第n列数据, 只接受整型参数。以kaggle上titanic data为例

```
women = train_data.loc[train_data.Sex == 'female']["Survived"]
```

这里的就是将train_data的Sex列为female的行取出, 赋值给women一个两列的数据, 第一列是train_data.Sex == 'female'对应的行数 (从0记), 第二列是对应的“Survived”值。

- 去除None值

就下面的两个语句清理Age列None值的效果是一样的

```
train_data_ageclean = train_data[train_data['Age'].notnull()]
test_data_ageclean = test_data.loc[test_data.Age.notnull()]
```

`.isnull()`则是取含None值的

```
df.fillna(0, inplace=True) #直接将NA替换成0
```

- `pd.get_dummies()`

将类别变量转换为虚拟/指标变量。

手册上的示例是将序列分类，以矩阵形式表现。

- `stack()`

`stack()`即“堆叠”，作用是将列旋转到行

`unstack()`即`stack()`的反操作，将行旋转到列

Xarray

[xarray数据结构](#)

BrainPy

- 计算基于 [JAX](#)

JAX

-  [谷歌 JAX 框架发展一览 | The Rise of JAX](#)

- [\[\]](#)

- [pytree](#)

[JAX101: 认识Pytrees数据结构](#)

在JAX中，使用 `pytree` 一词来指代由容器样的Python对象如list、dict、tuple、OrderedDict、None、namedtuple等）构建的类似树状结构。如果一些数据没有被python容器装起来，那么它就是子叶数据（比如数值、数组、类、字符串），`pytree`中可以嵌套`pytree`。

```
from jax.tree_util import tree_structure
a = (1, (2, 3), ()) # 3 leaves
print(tree_structure(a))
-----
> PyTreeDef((*, (*, *), ()))
```